

```
<style>.line { display: block; position: relative; padding-right: 160px; line-height: 1.6;}.line::after { content:
"LinkedIn · @AshishZope"; position: absolute; right: 0; bottom: 0; font-size: 9px; font-weight: 500; letter-
spacing: 1.1px; color: rgba(0, 0, 0, 0.5); white-space: nowrap; user-select: none; pointer-events: none; }</style>
<style> /* Modern Azure-themed header styling */ .modern-header { background: linear-gradient(135deg,
#0078D4 0%, #005A9E 50%, #003F7A 100%); color: white; text-align: center; font-size: 3em; font-weight: 700;
margin: 20px 0 10px 0; letter-spacing: 2px; font-family: 'Segoe UI', -apple-system, BlinkMacSystemFont, sans-
serif; text-shadow: 2px 2px 4px rgba(0,0,0,0.3); border-radius: 15px; padding: 30px 20px; box-shadow: 0 8px
32px rgba(0,120,212,0.3); position: relative; overflow: hidden; } .modern-header::before { content: ""; position:
absolute; top: 0; left: -100%; width: 100%; height: 100%; background: linear-gradient(90deg, transparent,
rgba(255,255,255,0.1), transparent); animation: shimmer 3s infinite; } @keyframes shimmer { 0% { left: -100%; }
100% { left: 100%; } } .modern-header .highlight { color: #00BCF2; text-shadow: 0 0 20px rgba(0,188,242,0.5);
position: relative; } .modern-header .subtitle { font-size: 0.4em; font-weight: 300; letter-spacing: 1px; margin-
top: 10px; opacity: 0.9; } /* Enhanced document styling */ body { font-family: 'Segoe UI', -apple-system,
BlinkMacSystemFont, 'Roboto', sans-serif; line-height: 1.6; color: #333; } h1, h2, h3, h4, h5, h6 { color: #0078D4;
font-weight: 600; margin-top: 30px; margin-bottom: 15px; } h2 { border-bottom: 3px solid #00BCF2; padding-
bottom: 10px; font-size: 1.8em; } h3 { color: #005A9E; font-size: 1.4em; } code { background-color: #f8f9fa;
border: 1px solid #e9ecef; border-radius: 4px; padding: 2px 6px; font-family: 'Consolas', 'Monaco', 'Courier
New', monospace; font-size: 0.9em; } pre { background-color: #f8f9fa; border: 1px solid #e9ecef; border-
radius: 8px; padding: 15px; overflow-x: auto; box-shadow: 0 2px 8px rgba(0,0,0,0.1); } pre code { background:
none; border: none; padding: 0; } table { border-collapse: collapse; width: 100%; margin: 20px 0; box-shadow: 0
2px 8px rgba(0,0,0,0.1); border-radius: 8px; overflow: hidden; } th, td { border: 1px solid #ddd; padding: 12px
15px; text-align: left; } th { background: linear-gradient(135deg, #0078D4, #005A9E); color: white; font-weight:
600; text-transform: uppercase; font-size: 0.9em; letter-spacing: 0.5px; } tr:nth-child(even) { background-color:
#f8f9fa; } tr:hover { background-color: #e3f2fd; transition: background-color 0.3s ease; } blockquote { border-
left: 4px solid #00BCF2; background-color: #f8f9fa; padding: 15px 20px; margin: 20px 0; border-radius: 0 8px
8px 0; font-style: italic; } .line { display: block; position: relative; padding-right: 160px; line-height: 1.6; }
.line::after { content: "LinkedIn · @AshishZope"; position: absolute; right: 0; bottom: 0; font-size: 9px; font-
weight: 500; letter-spacing: 1.1px; color: rgba(0, 0, 0, 0.5); white-space: nowrap; user-select: none; pointer-
events: none; } </style>
```

Pillars of OOP with real-time examples
(Opp's)

Pillars of OOP with real-time examples

4 Pillars of OOP with real-time examples

1. Encapsulation

Real-Time Example: ATM Machine

When you use an ATM, you can:

- Check balance
- Withdraw money
- Deposit money

But you **cannot directly access or modify the bank's database balance.**

This is **Encapsulation**: hiding data and providing controlled access through methods.

```
class BankAccount:
    def __init__(self):
        self.__balance = 50000

    def withdraw(self, amount):
        self.__balance -= amount

    def get_balance(self):
        return self.__balance

acc = BankAccount()
acc.withdraw(5000)

print(acc.get_balance())
```

Interview Statement

Just like an ATM hides the actual bank database and only allows operations through buttons, Encapsulation hides data and provides controlled access.

2. Inheritance

Real-Time Example: Employee Management System

All employees have:

- Name
- Employee ID

Managers have:

- Team size

Developers have:

- Programming language

Instead of writing common code repeatedly, child classes inherit from a parent class.

```
class Employee:
    def __init__(self, name):
        self.name = name

    def work(self):
        print("Employee works")
```

```
class Developer(Employee):  
    pass  
  
dev = Developer("Ashish")  
dev.work()
```

Interview Statement

Just like a Developer is an Employee and inherits common employee properties, Inheritance allows one class to acquire features of another.

3. Polymorphism

Real-Time Example: Payment Gateway

An e-commerce website can accept payments through:

- Credit Card
- UPI
- Net Banking

The action is the same: `pay()`

But implementation differs.

```
class CreditCard:  
    def pay(self):  
        print("Payment through Credit Card")  
  
class UPI:  
    def pay(self):  
        print("Payment through UPI")  
  
class NetBanking:  
    def pay(self):  
        print("Payment through Net Banking")  
  
payments = [CreditCard(), UPI(), NetBanking()]  
  
for p in payments:  
    p.pay()
```

Interview Statement

Just like different payment methods use the same `pay` option but process payments differently, Polymorphism allows the same method to behave differently.

4. Abstraction

Real-Time Example: Car Driving

When driving a car:

- You press the accelerator
- You use the steering wheel

You do **not need to know**:

- How the engine injects fuel
- How combustion happens
- How gears synchronize internally

Complex implementation is hidden.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Engine Started")

car = Car()
car.start()
```

Interview Statement

Just like a driver uses a car without knowing the engine's internal mechanism, Abstraction hides complexity and exposes only necessary features.

Complete Banking Example (All 4 Pillars Together)

Banking Application

OOP Pillar	Real-Time Banking Example
Encapsulation	Account balance is private
Inheritance	SavingsAccount inherits Account

OOP Pillar	Real-Time Banking Example
Polymorphism	Different accounts calculate interest differently
Abstraction	User sees Deposit/Withdraw, not backend calculations

```
from abc import ABC, abstractmethod

class Account(ABC):

    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    @abstractmethod
    def interest(self):
        pass

class SavingsAccount(Account):

    def interest(self):
        return self.get_balance() * 0.04

class CurrentAccount(Account):

    def interest(self):
        return self.get_balance() * 0.02

accounts = [
    SavingsAccount(100000),
    CurrentAccount(100000)
]

for acc in accounts:
    print(acc.interest())
```

FAANG Interview Answer (30 Seconds)

Consider an online banking system:

- **Encapsulation:** Account balance is kept private and accessed through methods.
- **Inheritance:** Savings Account and Current Account inherit common features from Account.
- **Polymorphism:** Different account types calculate interest differently using the same method name.

- **Abstraction:** Customers use deposit and withdrawal functions without knowing the internal banking logic.

These four concepts make software secure, reusable, scalable, and maintainable.

